Tropical Monte Carlo v3

Comprehensive Reference Manual

A numerical integration pipeline for generalized Euler integrals using tropical geometry and Monte Carlo sampling.

v3 = divergence/IBP engine (Tree A) ∪ lifting/complex engine (Tree B), refactored into a single non-duplicated package with complex-lifting fixes baked in.

Contents

1 Overview and Purpose

The **Tropical Monte Carlo v3** project is a numerical integration pipeline for evaluating generalized Euler integrals of the form:

$$I = \int_{[0,\infty)^n} dx_1 \cdots dx_n \prod_i x_i^{A_i} \prod_j P_j(\mathbf{x})^{B_j} \tag{1}$$

where:

- $P_j(\mathbf{x})$ are polynomials in $\mathbf{x}$ with real or complex coefficients, possibly depending on kinematic parameters.
- $A_i \in \mathbb{C}$ are the monomial exponents, possibly depending on a dimensional regulator ε.
- $B_j \in \mathbb{C}$ are the polynomial exponents, possibly depending on ε.

v3 unifies the capabilities of two predecessor codebases:

- **Tree A (`TROPICAL_MONTE_CARLO`):** divergence handling (tropical subtraction + IBP), batched VEGAS, factored C++ codegen. The base.
- **Tree B (`TROPICAL_MONTE_CARLOv2`):** auxiliary-variable lifting, complex exponents, `FlattenSector` refactoring, `SeedBase` runtime override. Grafted as Module 1b.
- **Bug-log complex-lifting branch (CALC2):** fixes for `SplitRealImag` mode (`MonoFactorLog`, complex `log P`), high-D fan K-scaling, high-D VEGAS sizing. Reconstructed from the bug logs as the genuinely new engineering of v3.

Exactness invariant (non-negotiable).

All tropical decomposition code is exact and symbolic. Floating-point arithmetic appears in exactly two places: (1) the numerical integration of flattened sector integrands (MC/VEGAS), and (2) independent numerical cross-checks (NIntegrate, CUBA). No decision inside the decomposition is made by evaluating a float.

Key innovation.

The code uses tropical geometry—the Newton polytope fan decomposition—to partition the integration domain into simplicial sectors. Within each sector, symbolic coordinate transformations and a "flattening" procedure render the integrand $\mathcal{O}(1)$ on $[0,1]^n$, making uniform Monte Carlo sampling highly efficient. A single C++ compilation handles thousands of kinematic points.

2 Mathematical Background

2.1 The Problem

Direct Monte Carlo on $[0,\infty)^n$ is impractical because the integrand has singular behavior near coordinate hyperplanes ($x_i \to 0$), different monomials dominate in different regions, and variance is enormous without importance sampling.

2.2 Tropical Geometry Approach

The tropical version of $P(\mathbf{x}) = \sum_m c_m \mathbf{x}^m$ replaces multiplication with addition and addition with min:

$$\text{trop}(P)(\mathbf{w}) = \min_m \{m \cdot \mathbf{w}\} \qquad (\mathbf{w} \in \mathbb{R}^n) \tag{2}$$

The **Newton polytope** $\text{NP}(P)$ is the convex hull of the exponent vectors. Its **normal fan** partitions $\mathbb{R}^n$ into cones, one per vertex, each corresponding to a region where one monomial dominates. Refining into simplicial cones gives the fan used here.

2.3 Why This Works

In each cone, the dominant monomial is factored out symbolically (an exact algebraic identity). After a monomial change of variables aligned with the cone's rays, the remaining polynomial has no singular behavior. A further "flattening" substitution absorbs the monomial singularity x^{A-1} into the measure, producing an $\mathcal{O}(1)$ integrand on $[0,1]^n$.

The fan depends only on the Newton polytope support (which monomials appear), not on coefficient values or the exponents B_j. For kinematic-dependent or extreme coefficients, compute the fan from a unit-coefficient proxy.

3 File Structure

```
TROPICAL_MONTE_CARLO3/
|
|-- SUMMARY.txt            Concise API + limitations reference
|-- tropical_fan.wl        Tropical fan computation via Polymake
|                          (+ computeFanScaled for high-D lifting)
|-- tropical_eval.wl       Main evaluation package (merged, v3)
|                          M0: Primitives
|                          M1: ProcessSector, FlattenSector
|                          M1b: Lifting
|                          M2: Divergence subtraction
|                          M2b: IBP pole extraction
|                          M3: Unified C++ codegen
|                          M4: Unified driver
|                          M5: Validation
|
|-- EXAMPLES/              Worked examples and tests
|-- TEST/                  43 named cross-checks + baselines/
|-- SANDBOX/               Lifting development experiments
|-- CC/                    FIESTA cross-checks (optional dep.)
|-- INTERFILES/            Generated C++, binaries (gitignored)
|-- MANUAL/                This manual
```

4 Architecture: The 5-Feature Pipeline

4.1 Stage 1: Tropical Fan Decomposition (`tropical_fan.wl`)

Input:

Newton polytope of $P_1 \cdots P_m$ (unit-coefficient proxy for kinematic or extreme coefficients)

Process:

Convex hull $\rightarrow$ normal fan $\rightarrow$ simplicial triangulation (via Polymake)

Output:
`{dualVertices, simplexList}`—integer ray generators plus simplicial cone n-tuples

v3 addition:
Private `computeFanScaled` retries on $K\cdot$verts with $K \in \{1, n+2, 2n+4, 6n+6\}$ (scale-invariant normal fan) to handle thin-simplex failures in dim ≥ 4.

4.2 Stage 2: Symbolic Sector Processing

Source:
Module 1—`ProcessSector`, `FlattenSector`

Process:
For each simplicial cone: (a) build ray matrix M, (b) monomial change of variables, (c) tropical factoring, (d) effective exponents, (e) convergence gate, (f) flattening

Output:
`SectorData` association per cone (see Section 7)

`FlattenSector` is factored out (from Tree B) and accepts an optional `eps` argument: the divergence predicate is evaluated at $\varepsilon \to 0$ when `eps` is present, enabling the lifted path to consume pre-flattening `ClearedPolys` before its own delta resolution.

4.3 Stage 3a: Divergence Regulation

Source:
Module 2 (subtraction) + Module 2b (IBP)

When:
Any sector has $\mathrm{Re}(a_{\mathrm{eff},k})|_{\varepsilon=0} \leq 0$

Limitation:
Single divergent variable per sector only. Nested divergences ($1/\varepsilon^{d\geq 2}$) refused cleanly (`$Failed`). See Section 9.

Both methods are kept; IBP is the default (`Method -> Automatic` $\Rightarrow$ `"IBP"`); subtraction is the validating cross-check.

4.4 Stage 3b: Auxiliary-Variable Lifting

Source:
Module 1b—`DetectExtremeCoefficients`, `LiftCoefficients`, `ProcessSectorLifted`

When:
Polynomial coefficient $|C| \gg 1$ or $|C| \ll 1$ inflates MC variance

Composes with divergence (Case A):
a lifted sector may *also* carry a single $1/\varepsilon$ pole. After the lifting δ-resolution collapses the auxiliary dimension, a lifted sector is structurally an ordinary n-dimensional `SectorData` whose pole lives in one of the surviving effective exponents $\tilde{a}_j$—the object Stage 3a already consumes. `ProcessSectorLifted` is ε-aware and routes through the divergence machinery (Section 5.7.4). Complex polynomial exponents B compose too, via `ComplexExponentMode->"SplitRealImag"` on both routes (planCXLIFTDIV.md, cross-check 46). Refused cases: the divergent variable coupling to the lifted domain face (Case B), `SplitRealImag`×lift×divergence on the *inline symbolic-*ε Subtraction path, and `Direct`-mode complex-B lifting.

4.5 Stage 4: C++ Code Generation and Execution

Source:
Module 3—`GenerateCpp`, `CompileCpp`, `detectCuba`

Process:
Translate Mathematica expressions → C++; generate integrand functions; generate `main()` (MC or VEGAS); compile; run; read results.

Samplers:
`"MC"` (default, plain uniform Monte Carlo) or `"VEGAS"` (CUBA-Vegas, opt-in; requires the CUBA library). Hard `$Failed` if VEGAS requested but CUBA absent.

5 Core Algorithms

5.1 Sector Decomposition by Monomial Maps

For a simplicial cone σ with rays $\{\rho_1, \ldots, \rho_n\}$, define

$$M = -\text{Transpose}[\rho_1, \ldots, \rho_n], \qquad x_i = \prod_j y_j^{M_{ij}}, \quad \mathbf{y} \in (0,1]^n. \tag{3}$$

The Jacobian and monomial prefactor combine to give raw exponents:

$$\text{rawA}_j = \sum_i (A_i + 1)\, M_{ij}, \tag{4}$$

and the sector integral reads $I_\sigma = |\det M| \int_{[0,1]^n} d^n y \ \prod_j y_j^{\text{rawA}_j - 1} \prod_k P_k(\mathbf{y}^M)^{B_k}$.

5.2 Tropical Factoring (exact)

For each polynomial P_k, define $d_{k,j} = \min_m\{e_{m,j}\}$ (min over all monomials) and factor:

$$P_k(\mathbf{y}) = \mathbf{y}^{d_k} \cdot Q_k(\mathbf{y}). \tag{5}$$

This is an exact algebraic identity for any coefficient values. The cleared polynomial Q_k has all non-negative exponents and contains a constant term (the vertex-dual dominant monomial), giving $Q_k(\mathbf{y}) \geq c_{\text{dom}} > 0$ on $[0,1]^n$.

5.3 Effective Exponents and Convergence Gate

$$a_{\text{eff},j} = \text{rawA}_j + \sum_k B_k\, d_{k,j} \in \mathbb{C}. \tag{6}$$

A sector is convergent if $\text{Re}(a_{\text{eff},j}) > 0$ for all j (evaluated at $\varepsilon = 0$ when a regulator is present). The imaginary parts of A_i and B_j contribute to a_{eff} but do not affect convergence.

5.4 Flattening

$$y_j = (y_j')^{1/a_j} \implies y_j^{a_j - 1}\, dy_j = \frac{1}{a_j}\, dy_j'. \tag{7}$$

The y^{a-1} factor is completely absorbed. The sector integral becomes:

$$\boxed{I = \sum_\sigma \frac{|\det M_\sigma|}{\prod_j a_j^\sigma} \int_{[0,1]^n} d^n y' \ \prod_k Q_k^\sigma(\mathbf{y}'^{1/a^\sigma})^{B_k}} \tag{8}$$

For complex a_j, this is the analytic continuation; geometrically, $\text{Im}(a_j) \neq 0$ rotates the integration contour onto a logarithmic spiral and additionally suppresses MC variance. See Figure 1.

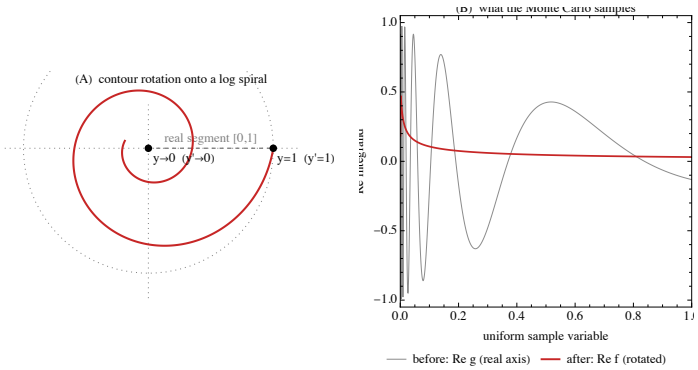

Figure 1: Complex flattening contour rotation. When a_j has a nonzero imaginary part, the substitution $y_j = (y'_j)^{1/a_j}$ rotates the sector's integration contour onto a logarithmic spiral in the complex plane. This suppresses the oscillatory contributions from $B_k \in \mathbb{C}$ and can dramatically reduce Monte Carlo variance. Reproduced from Tree B `MANUAL/complex_flatten_spiral.pdf`.

5.5 Divergence Regulation: Tropical Subtraction (Module 2)

When $a_k(\varepsilon) = c_k\varepsilon + \mathcal{O}(\varepsilon^2)$ with $c_k > 0$:

$$I_{\text{sector}} = \frac{G_0}{c_k\varepsilon} + \frac{G_1}{c_k} + R + \mathcal{O}(\varepsilon) \tag{9}$$

where:

G_0**:** $(n-1)$-dim integral with $y_k = 0$ (pole coefficient).

G_1**:** G_0 integrand times log-insertion sum $\sum_i \frac{a_i^{(1)}}{a_i^{(0)}} \log y'_i + \sum_j B_j^{(1)} \log Q_j|_{y_k=0}$.

R**:** Remainder $\int [F_{\text{full}} - F_{\text{simple}}] \cdot y_k^{a_k^{(0)}-1}\, d\mathbf{y}$ (convergent at $\varepsilon = 0$).

The analytic pole $1/c_k$ is computed symbolically; G_0, G_1, R are evaluated numerically.

5.6 Divergence Regulation: IBP (Module 2b, default)

Integration by parts on $[0,1]$ for the divergent variable y_k:

$$\int_0^1 y_k^{a_k-1} f\, dy_k = \frac{1}{a_k}\Big[f\big|_{y_k=1} - \int_0^1 y_k^{a_k} \partial_{y_k} f\, dy_k\Big] \tag{10}$$

$\partial_{y_k} \prod_j Q_j^{B_j}$ decomposes into monomial terms, each with shifted exponents $\alpha_{t,k} = a_k + e_{m,k} \geq 1$ at $\varepsilon = 0$ (convergent). The analytic pole structure:

$$\text{Pole:} \quad \frac{B^{(0)} - S_0}{c_k\varepsilon} \tag{11}$$

$$\text{Finite:} \quad \frac{(B^{(1)} - S_1) - r_k(B^{(0)} - S_0)}{c_k} \tag{12}$$

where $S_0 = \sum_t \text{coeff}_t^{(0)} I_{t,0}$, $r_k = a_k^{(2)}/c_k$. IBP and subtraction agree on (pole, finite) to $\lesssim 10^{-3}$ (cross-check #4); this agreement is one of the strongest correctness signals in v3.

5.7 Auxiliary-Variable Lifting (Module 1b)

Replace the extreme coefficient C in monomial $C\mathbf{x}^\alpha$ by $z^k\mathbf{x}^\alpha$, with anchor $z_0 = |C|^{1/k}$ (kept exact in Mathematica) and residual $c = C/z_0^k$ (the $\mathcal{O}(1)$ phase or sign). Insert $\int dz\,\delta(z - z_0)$. The (n+1)-dimensional lifted integrand is decomposed on the fan of the lifted polynomial; the delta constraint is resolved per sector by a pivot substitution.

5.7.1 Anchor Selection (k^* rule)

The anchor $z_0 = |C|^{1/k}$ and the integer k are locked together: k must be a positive integer (it is the z-exponent in the lifted monomial and a lattice coordinate of the lifted Newton polytope), so choosing k determines z_0 completely. The optimal stopping point is self-consistency with the detector: the package already flags a coefficient as extreme when it leaves the band $[1/\tau, \tau]$ (threshold τ, default 10^3), so pick the *smallest* k that brings the anchor back into the band:

$$k^* = \max\Big(1,\ \Big\lceil \frac{|\log|C||}{\log\tau} \Big\rceil\Big) \xrightarrow{\tau=10^3} \max\Big(1,\ \Big\lceil \frac{\log_{10}|C|}{3} \Big\rceil\Big). \tag{13}$$

Going past k^* buys only sub-threshold residual shrinkage while paying geometry cost (inflated $|\det M|$, shrunken integrability margins, more `EmptyDomain` sectors). The same formula covers both large and small coefficients (it uses $|\log|C||$). Worked cases from the v3 benchmarks:

Case	$\lvert C\rvert$	k^*	$z_0 = \lvert C\rvert^{1/k^*}$	hand-picked k
Case A ($10^6 x_1^2$)	10^6	2	10^3	2
Case B primary ($10^4 x_1^2$)	10^4	2	10^2	2
Example 20 ($10^{-4} x_1^2$)	10^{-4}	2	10^{-2}	2

Decision order.

(1) Geometry gate first: require a surviving sector with `HasConstantTerm=True` and all $\mathrm{Re}[\tilde{a}] > 0$; if no such sector exists for any k, **do not lift**. (2) Smallest k with $z_0 \in [1/\tau, \tau]$ (i.e. k^*). (3) Simpler geometry (lower $|\det M|$, fewer empty sectors). The full procedure, including the multi-rule multi-coefficient case, is in the file `AUXT/anchor_selection_procedure.md` under `OLD_CODE/TROPICAL_MONTE_CARLO2/TROPICAL_MONTE_CARLOv2/`.

5.7.2 Pivot Selection and Delta Elimination

For a simplicial cone with (n+1)-dimensional sector data, let $m = (m_1, \ldots, m_{n+1})$ be the z-row of the ray matrix. Select a pivot p with $m_p \neq 0$. The delta constraint gives:

$$y_p^* = z_0^{1/m_p} \prod_{j \neq p} y_j^{-m_j/m_p} \tag{14}$$

Substituting this in the cleared polynomial, re-clearing (Step 2 repeated), and absorbing the delta Jacobian produces an n-dimensional sector with effective exponents:

$$\tilde{a}_j = a_j - a_p \frac{m_j}{m_p} + \sum_k B_k \tilde{d}_{k,j}, \qquad \text{prefactorBase} = \frac{|\det M|}{|m_p|} z_0^{a_p/m_p - 1}. \tag{15}$$

Pivot ranking (constant-term preservation first):

1. `HasConstantTerm = True` after re-clearing. *First because sectors without a constant term can have infinite true variance (L5 risk, Section 9).*
2. $|m_p| = 1$ (unit pivot exponent).
3. $\max \min_j \mathrm{Re}[\tilde{a}_j]$ (best integrability margin).

Domain constraint.

The lifted sector only contributes where $y_p^* \in (0,1]$, i.e. $\log y_p^* \le 0$. Sign analysis classifies sectors as: always feasible (no indicator), never feasible (`EmptyDomain`, returned and dropped), or mixed. The domain-constraint indicator in the flattened variables $y'_j = (y_j)^{\tilde{a}_j}$ is:

$$\log y_p^* = \frac{1}{m_p}\Big(\log z_0 - \sum_{j\neq p} \frac{m_j}{\tilde{a}_j} \log y'_j\Big) \le 0, \tag{16}$$

compiled into the C++ integrand (returns 0 outside the feasible region).

5.7.3 `HasConstantTerm = False`: the L5 Caveat

When the re-cleared polynomial $\tilde{Q}$ lacks a constant term, the lifted integrand may not be in L^2 on $[0,1]^n$. The sample-based standard error understates the true error; the MC estimate can significantly deviate from the true value. Always cross-check against an independent reference (NIntegrate, CUBA Cuhre) rather than trusting the MC error bar for `HasConstantTerm=False` sectors.

5.7.4 Lifting a Divergent Integral (Case A)

Lifting and a simple $1/\varepsilon$ pole compose. The two features act on different objects: lifting acts on a polynomial *coefficient* (a +1-dimensional δ-constraint), while divergence acts on a monomial *exponent* $a_k(\varepsilon) \to 0$ (a $1/\varepsilon$ endpoint pole). After the δ-resolution collapses the auxiliary dimension, the pole—if any—necessarily sits in one of the n post-δ effective exponents $\tilde{a}_j$, exactly the object the IBP/subtraction machinery consumes.

ε-aware classification.

`ProcessSectorLifted[..., "Eps"->eps]` classifies each $\tilde{a}_j$ at $\varepsilon \to 0$ *exactly* (`PossibleZeroQ[Im]` and `Re[...]>0`, never a tolerance): `"complex"` ($\to$ `liftcomplex`), `"convergent"` (Re > 0), or `"divergent"` (Re ≤ 0, regulator present). A pivot is admissible iff no direction is complex and *at most one* is divergent (with $c_k = \partial_\varepsilon \tilde{a}_k|_0 \neq 0$). With `"Eps"->None` the classification reduces exactly to the pre-ε realness test, so convergent lifting is byte-identical.

Prefactor ε-dependence.

A divergent lifted `SectorData` carries `"PrefactorBase"` $= (|\det M|/|m_p|)\, z_0^{a_p/m_p - 1}$, which itself carries ε through $a_p(\varepsilon)$. The divergence routines use `PrefactorBase`$|_{\varepsilon=0}$ in the integrand and the IBP Laurent assembly adds the missing finite term $(\partial_\varepsilon \log \text{PrefactorBase})|_0 \cdot (\text{pole})$, stored as `"DLogPrefactor"` ($= 0$ for unlifted sectors).

Case A vs Case B.

The lifted domain indicator has coefficients $\text{ic}_j = m_{\text{other},j}/\tilde{a}_j$. *Case A* (supported): the divergent slot decouples (`DomainConstraint === None` or $\text{ic}_k = 0$, i.e. $m_{\text{other},k} = 0$); the indicator commutes with the y_k IBP/subtraction and rides along on the other coordinates. *Case B* (refused, `liftdivdomain`): $\text{ic}_k \neq 0$ for *every* admissible pivot. The pivot ranking adds a *Case-A-first* key so a decoupled pivot wins whenever one exists; Case B is refused only when none does (rare—no trigger found across a diverse $n = 2, 3, 4$ battery).

Two routes.

Both Stage-3a routes resolve the lifted Laurent (cross-checks 41-C and 45, $n = 2, 3, 4$, non-trivial polynomials). `LaurentFromSubtraction` (pin ε, sum the full decomposition, fit) is the *robust universal* route—it does not rely on per-sector pole classification, so it is correct even when non-trivial polynomials produce `HasConstantTerm=False` sectors. The per-sector `Method->"IBP"`

route is exact only when every sector is `HasConstantTerm`-clean; otherwise a convergent sector can carry a polynomial-zero pole ($\tilde{Q}_j \to 0$, $B_j < 0$) that the $\tilde{a}$-classification misses and the IBP pole undercounts (a pre-existing sector-decomposition limitation, not specific to lifting). Use the Subtraction route there.

5.8 Complex Exponents in Codegen

The default complex-exponent path (`ComplexExponentMode -> "Direct"`) evaluates:

$$\texttt{result *= std::exp(B\_j * std::log(P\_j));} \quad \text{(complex } \texttt{log}\text{)} \tag{17}$$

This is always correct. For polynomial exponents $B_j \in \mathbb{C}$ and polynomials with complex coefficients ($\arg P \neq 0$), the complex `log` is essential.

The opt-in `"SplitRealImag"` VEGAS-variance mode uses the split identity:

$$\prod_j P_j^{B_j} = \prod_j P_j^{\mathrm{Re}(B_j)} \cdot \exp\Big(i \sum_j \mathrm{Im}(B_j) \log P_j\Big) \tag{18}$$

Two fixes are required before this mode is correct:

Fix A (BUG 1, `TMCv2_BUG_LOG`).

Use the complex `log`. The oscillatory phase $\exp(i\,\mathrm{Im}(B)\log P)$ splits as $\exp(-\mathrm{Im}(B)\arg P) \cdot \exp(i\,\mathrm{Im}(B)\log|P|)$; using `std::log(std::abs(P))` silently drops the factor $\exp(-\mathrm{Im}(B)\arg P)$, which equals 1 only for real-positive P. With complex coefficients $\arg P \neq 0$, so this bug is invisible in all real-coefficient tests but wrong in general.

Fix B (L3, `lift_error_log`).

Restore the tropical monomial factor. Tropical factoring removed the monomial $\mathbf{y}^{d_k}$; the oscillatory phase must add back:

$$\sum_k \mathrm{Im}(B_k)(\mathrm{Const}_k + \sum_i \mathrm{Coeffs}_{k,i} \log y'_i), \tag{19}$$

where for unlifted sectors $\mathrm{Const} = 0$, $\mathrm{Coeffs}_i = d_{k,i}/a_{\mathrm{eff},i}$; for lifted sectors the residual exponents include contributions from the pivot substitution. Stored as `SectorData["MonoFactorLog"] = {Const, Coeffs}`.

`SplitRealImag` caveat.

For very large $\mathrm{Im}(B)$, VEGAS importance sampling built on $\mathrm{Re}(B)$ can inflate variance. Always compare `SplitRealImag` against `Direct` and an independent reference before trusting `SplitRealImag` results. The default `"Direct"` mode is always safe.

5.9 High-Dimensional VEGAS Sizing (`resolveVegasSizing`)

VEGAS defaults (`NStart=1000, NIncrease=500, NBatch=1000`) are tuned for $n \leq 4$. For sector dimension $d \geq 6$ they return confidently wrong values with deceptively tight error bars (measured: 8D unlifted 1.2% off at 7σ; lifted $\sim$45% off). The fix: resolve `Automatic` via `NStart` $\approx \mathrm{base} \cdot 3^{d-4}$ for $d \geq 5$; fire `TropicalEval::vegasbudget` when `NSamples`$< 20 \cdot$`NStart`. In high dimensions, always cross-check against an independent reference rather than trusting the VEGAS error bar.

6 Core File Reference

6.1 `tropical_fan.wl`

`PolytopeVertices[integrand, vars]` Extracts Newton polytope lattice points from the integrand.

`ComputeDecomposition[vertices]` Returns `{dualVertices, simplexList}`. Internally calls the private `computeFanScaled` for automatic K-scaling retry on degenerate thin-simplex cases.

`computeFanScaled[vertices]` *(private)* The K-scaling fan helper in `tropical_fan.wl`: retries the normal-fan computation on $K \cdot$verts with $K \in \{1, n+2, 2n+4, 6n+6\}$ (scale-invariant normal fan) to recover from thin-simplex failures in dim ≥ 4. Used both by `ComputeDecomposition` and by `EvaluateTropicalMCLifted`'s automatic `"FanData"` path (Section 6.7).

`ComputeDecompositiony[vertices]` Variant intersecting with the positive orthant.

`convexHullVertices[points]` Calls Polymake to compute the convex hull.

`translateToOriginInteger[vertices]` Shifts polytope so the origin is an interior lattice point.

`$TropicalFanPolymakePath` Override Polymake auto-detection before loading.

`$SkipPolymakeLoad` Set to `True` before loading to skip `tropical_fan.wl` (for cluster nodes without Polymake; supply `fanData` explicitly).

The fan properties required by the evaluation algorithm: (1) simpliciality ($\det \neq 0$; violations caught by `degenerate`/`notsimplicial`), (2) completeness ($\bigcup_\sigma = \mathbb{R}^n$), (3) essential disjointness, (4) vertex duality (one monomial of each P_k attains the tropical minimum throughout the cone—guarantees the constant term in Q_k).

6.2 `tropical_eval.wl` — Module 1: `ProcessSector` and `FlattenSector`

```
ProcessSector[integrandSpec, dualVertices, simplex, coneIndex]
FlattenSector[clearedPolys, effectiveAVals, prefactorBase, eps:None]
```

`ProcessSector` executes Steps 1–3 (monomial map, tropical factoring, effective exponents) and calls `FlattenSector` for Step 4 (flattening). When `eps` is passed, the divergence predicate evaluates effective exponents at $\varepsilon \to 0$ via:

```
a0 = If[eps =!= None, effectiveAVals /. eps -> 0, effectiveAVals];
isDivergent = Or @@ (TrueQ[Re[#] <= 0] & /@ a0);
```

The `IsDivergent -> True` pre-flattening early-return branch of `ProcessSector` is preserved: the lifted path consumes `ClearedPolys`/`NewExponents` from it, because $(n{+}1)$-dim pre-delta sectors are routinely flagged divergent even when the delta-constrained integral is finite.

`CheckFlatteningMagnitude[sectorData, nSamples].`

Evaluates the flattened integrand at random points in $[0,1]^n$, reporting min/max/mean magnitude and warning when max $> 10^3$ or min $< 10^{-6}$ (a heavy-tail indicator). For lifted sectors with a `DomainConstraint`, draws feasible points by rejection sampling and additionally returns `"FeasibleFraction"`.

6.3 `tropical_eval.wl` — Module 1b: Lifting

`DetectExtremeCoefficients[spec, threshold:1000, opts]` Returns a list of associations `{<|"PolyIndex","ExponentVector","Coefficient","Magnitude","SuggestedK"|>}` for every monomial with $|C| < 1/\tau$ or $|C| > \tau$. `SuggestedK` uses the k^* rule (Section 5.7.1). Symbolic coefficients skipped.

`LiftCoefficients[spec, liftRules]` Applies lifting: each rule is `<|"PolyIndex"->j,"ExponentVector"->`. Returns `<|"LiftedSpec"->...,"LiftData"->...|>`. Performs the round-trip identity check ($z \to z_0$ recovers the original polynomial exactly); fires `liftidentity` on failure.

`ProcessSectorLifted[liftedSpec, dualVerts, simplex, coneIndex, liftData]` Runs `ProcessSector` in dimension $n{+}1$, then resolves the delta constraint (pivot selection, substitution, re-clearing, domain analysis). Returns `SectorData` extended with lifted keys. Error paths: `liftnopivot`, `liftcomplex`, `liftdivergent`.

`ValidateLiftedDecomposition[origSpec, liftedSpec, liftedFan, liftData, kin, pg]` Cross-checks the lifted sector sum against direct `NIntegrate` of the original integral. Returns `DirectResult`, `SectorSum`, `RelativeError`, `SectorResults`, `DroppedSectors`.

6.4 `tropical_eval.wl` — Module 2: Tropical Subtraction

`IdentifyDivergences[sectorData, eps]` Expands effective exponents in ε, finds variables with $\mathrm{Re}(a_k(0)) \leq 0$. Returns divergent variable index, c_k, $a^{(0)}$, $a^{(1)}$.

`ProcessDivergentSector[sectorData, integrandSpec]` Constructs G_0, G_1, Remainder. Returns `DivergentSectorData`.

`ValidateSubtraction[divSD, sd, spec, kin, eps]` Cross-checks $G_0/(c_k\varepsilon) + G_1/c_k + R$ against direct `NIntegrate` at finite ε.

6.5 `tropical_eval.wl` — Module 2b: IBP (default method)

`IBPReduceSector[sectorData, eps]` Symbolic IBP decomposition: identifies all divergent variables and iteratively applies IBP to produce convergent terms. Returns `IBPPrefactors`, `Terms`, `DivergentVariables`, `NTerms`.

`IBPProcessSector[sectorData, integrandSpec]` Full pipeline: reduction + ε-expansion + per-term flattening + boundary construction. Returns `IBPSectorData`.

`IBPCheckBoundary[sectorData, integrandSpec, nPoints]` Verifies IBP boundary terms numerically.

`ValidateIBP[ibpSD, sd, spec, kin, eps]` Cross-checks IBP formula against `NIntegrate` at finite ε. Returns `RelativeError`.

6.6 `tropical_eval.wl` — Module 3: Unified C++ Codegen

`MmaToC[expr, paramMap].`

Recursive converter Mathematica → C++:

Mathematica	**C++**
Integer n	`n.0`
Rational p/q	`(p.0/q.0)`
`Complex[a,b]`	`cx(a, b)`
`Power[x, -1]`	`(1.0/x)`
`Power[x, 1/2]`, `Power[x,-1/2]`	`std::sqrt(x)`, `(1.0/std::sqrt(x))`
`Power[E, z]`, `Exp[z]`	`std::exp(z)`
`Power[x, e]`	`std::pow(x, e)`
`Log[x]`	`std::log(x)`
`Re[z]`, `Im[z]`	`(z).real()`, `(z).imag()`
Kinematic symbol	`params[i]`

The `badcpp` gate: after generation, if any emitted code contains unconverted Mathematica heads or `ComplexInfinity`/`Indeterminate`, `GenerateCpp` returns `$Failed` (`TropicalEval::badcpp`) instead of writing uncompilable C++.

Polynomial evaluation uses the **log-exp trick**:

$$c \cdot y_1^{\alpha_1} \cdots y_n^{\alpha_n} = c \cdot \exp(\alpha_1 \log y_1 + \cdots + \alpha_n \log y_n), \tag{20}$$

which is branch-cut free for $y_i \in (0, 1]$ and works for complex α_i.

`GenerateCpp[convSectors, divSectors, spec, outputFile, opts].`

Generates a complete, self-contained C++ source file with integrand functions of all types:

`integrand_conv_N`: Convergent sector. For lifted sectors, includes the domain-constraint indicator (returns 0 outside the feasible region).

`integrand_g0_N`: G_0 integral ($(n-1)$-dim, $y_k = 0$ simplified polys).

`integrand_g1_N`: G_1 integral ($G_0\times$ log-insertion sum).

`integrand_rem_N`: Remainder (prefactor $\times\, y_k^{a_k^{(0)}-1} \times [F_{\text{full}} - F_{\text{simple}}]$).

IBP types: Boundary and term integrands, parametrized by the integrand-type tag.

The `main()` reads kinematic points from an input file, loops over points (OpenMP-parallelized for MC; serial for VEGAS to avoid CUBA thread-safety issues), draws samples, accumulates mean and variance with Welford's online algorithm, and writes one line per kinematic point: `re im err_re err_im`.

Sampler options (Table 1).

Batch mode (`"Batch" -> True`).

VEGAS only, convergent/lifted-convergent path only. Shares one low-discrepancy sample set across a chunk of kinematic points (up to `CubaMaxComp`/2 points per chunk for complex integrands; hard ceiling to avoid CUBA segfault). Divergent/IBP path falls back to per-kp VEGAS. Typically 30–60× wall-clock speedup on large scans.

Option	Default	Description
`"Integrator"`	`"MC"`	`"MC"` (plain uniform) or `"VEGAS"` (CUBA-Vegas)
`"Batch"`	`False`	VEGAS only: share one LDS sample set across kinematic chunk
`"VegasEpsRel"`	`Automatic`	Vegas relative tolerance (tiny default makes maxeval binding)
`"VegasEpsAbs"`	`Automatic`	Vegas absolute tolerance
`"VegasSeed"`	`0`	CUBA seed; 0 = Sobol (deterministic)
`"VegasNStart"`	`Automatic`	Resolved by `resolveVegasSizing`
`"VegasNIncrease"`	`Automatic`	Resolved by `resolveVegasSizing`
`"VegasNBatch"`	`Automatic`	Resolved by `resolveVegasSizing`
`"VegasMinEval"`	`0`	CUBA mineval
`"CubaMaxComp"`	`512`	Batch-mode `ncomp` ceiling
`"NSamples"`	`10^6`	samples/sector for MC; maxeval/sector for VEGAS
`"SeedBase"`	`42`	Per-kp RNG seed base; also runtime `argv[5]` override

Table 1: Sampler options on `GenerateCpp`, `CompileCpp`, and `EvaluateTropicalMC`.

CUBA detection and threading.

`detectCuba[]` is memoized and detects the `TROPICAL_REQUIRES_CUBA` sentinel in the generated source. If VEGAS is requested but CUBA is absent, compilation fails fast with an actionable message (`TropicalEval::nocuba`); no silent MC fallback. CUBA is not thread-safe across concurrent calls; Vegas mains loop serially with `cubacores(0,0)`. To use many cores on a large scan, shard the kinematic file across processes or use batch mode.

6.7 `tropical_eval.wl` — Module 4: Unified Driver

```
EvaluateTropicalMC[integrandSpec, fanData, kinematicPoints, opts...]
```

Routes on `"Method"` (∈ `{Automatic, "None", "IBP", "Subtraction"}`) and `"LiftData"` (∈ `{None, <association from LiftCoefficients>}`). The 10-step skeleton is shared: workdir → fan unpack → per-sector processing → optional Validate → codegen → compile → run → readback → assemble → error summary.

Thin wrappers for API continuity:

`EvaluateTropicalMCLifted[spec, kin, opts]` Auto-detects/applies lifting and routes through the unified driver via the `"LiftData"` option (see Section 6.7). This is the typical user-facing entry point for lifting; the bare `"LiftData"` option on `EvaluateTropicalMC` is the advanced/low-level path.

`LaurentFromSubtraction[spec, fan, kin, opts]` Forces `"Method" -> "Subtraction"` and returns the Laurent expansion as `<|-1 -> pole, 0 -> finite|>`.

`EvaluateTropicalMCIBP[spec, fan, kin, opts]` Superseded by `EvaluateTropicalMC[..., "Method" -> "IBP"]` (which routes to the same private driver); kept for backward compatibility.

Full option list.

Option	Default	Description
`"Method"`	`Automatic`	`"None"` | `"IBP"` | `"Subtraction"`; Auto ⇒ IBP if divergent
`"LiftData"`	`None`	Association from `LiftCoefficients[]` or `None`; advanced/low-level
`"Integrator"`	`"MC"`	`"MC"` or `"VEGAS"`
`"EpsilonValue"`	`None`	Substitute $\varepsilon \to$ value (all sectors become convergent)
`"ComplexExponentMode"`	`Automatic`	`"Direct"` or `"SplitRealImag"` (see Section 5.8)
`"NSamples"`	`10^6`	Samples per sector
`"NThreads"`	`Automatic`	OpenMP threads (`$ProcessorCount`)
`"SeedBase"`	`42`	RNG seed base
`"RunChecks"`	`True`	Symbolic Validate* gating before the run
`"PrecisionGoal"`	`3`	Precision goal for validation integrals
`"Verbose"`	`True`	Progress output
`"WorkingDirectory"`	`Automatic`	Where generated files go (`INTERFILES/`)

Plus all sampler options from Table 1.

`EvaluateTropicalMCLifted` signature and options.

```
EvaluateTropicalMCLifted[integrandSpec, kinematicPoints, opts...]
```

The thin lifting wrapper: it detects/applies lifting (calling `DetectExtremeCoefficients` then `LiftCoefficients` internally), builds the $(n{+}1)$-dimensional lifted fan, and routes through `EvaluateTropicalMC` with the resulting `"LiftData"`. With no extreme coefficients it falls back to plain `EvaluateTropicalMC` on the original spec. Its five own options (it also forwards all `EvaluateTropicalMC` options):

Option	Default	Description
`"LiftRules"`	`Automatic`	Auto-detect extreme coefficients via `DetectExtremeCoefficients`, or supp
`"Threshold"`	`1000`	Extreme-coefficient magnitude threshold passed to `DetectExtremeCoeffic`
`"AnchorRule"`	`"kStar"`	Anchor $z_0 = \lvert C\rvert^{1/k}$ selection (`"kStar"` default; `"Unit"` = legacy)
`"BandEdgeGuard"`	`False`	Band-edge guard in extreme-coefficient detection
`"FanData"`	`Automatic`	Lifted fan; Automatic uses the K-scaled robust path (`computeFanScaled`)

6.8 `tropical_eval.wl` — Module 5: Validation

`ValidateDecomposition[spec, fanData, testKin, pg]` Sector sum vs. direct `NIntegrate`. Returns `DirectResult, SectorSum, RelativeError, SectorResults`.

`ValidateLiftedDecomposition[origSpec, liftedSpec, liftedFan, liftData, kin, pg]` Lifted sector sum vs. direct `NIntegrate` of the original integral. Returns `DroppedSectors`.

`ValidateSubtraction[divSD, sd, spec, kin, eps]` $G_0/(c_k\varepsilon) + G_1/c_k + R$ vs. `NIntegrate` at finite ε.

`ValidateIBP[ibpSD, sd, spec, kin, eps]` IBP formula vs. `NIntegrate`. Returns `RelativeError`.

`RunAllTests[]` Full v3 suite entry point; exits nonzero on any FAIL.

7 Data Structures

7.1 `IntegrandSpec`

Key	Description
`"Polynomials"`	$\{P_1, \ldots\}$—symbolic polynomial expressions; coefficients may be complex or kinematic-dependent.
`"MonomialExponents"`	$\{A_1, \ldots, A_n\}$—one per variable; may be complex or ε-dependent.
`"PolynomialExponents"`	$\{B_1, \ldots, B_m\}$—one per polynomial; typically negative; may be complex or ε-dependent.
`"Variables"`	$\{x_1, \ldots, x_n\}$—integration variable symbols.
`"KinematicSymbols"`	$\{\lambda, \mu, \ldots\}$ or $\{\}$.
`"RegulatorSymbol"`	ε or `None`.

7.2 `SectorData` (from `ProcessSector`)

Key	Description
`"ConeIndex"`	Integer label for this simplicial cone.
`"RayMatrix"`	M $(n \times n)$.
`"DetM"`	$\det(M)$.
`"SelectedRays"`	$\{\rho_1, \ldots, \rho_n\}$.
`"RawExponents"`	rawA: exponents after M-transformation, before tropical factoring.
`"NewExponents"`	a_{eff}: effective exponents after tropical factoring.
`"MinExponents"`	$\{d_{k,j}\}$ per polynomial.
`"TransformedPolys"`	Monomials in $\mathbf{y}$ (exponents may be negative).
`"ClearedPolys"`	After tropical factoring; all exponents ≥ 0.
`"FlattenedPolys"`	After flattening; exponents $e_{m,i}/a_{\text{eff},i}$.
`"Prefactor"`	$\lvert\det(M)\rvert / \prod_i a_{\text{eff},i}$.
`"PrefactorBase"`	**[v3 new]** Sector prefactor *before* dividing by $\prod_i a_{\text{eff},i}$: $\lvert\det(M)\rvert$ unlifted, $(\lvert\det M\rvert/\lvert m_p\rvert) z_0^{a_p/m_p - 1}$ lifted. The divergence routines reconstruct prefactors from this (Section 5.7.4); unlifted output is byte-identical.
`"IsDivergent"`	`True/False`.
`"DivergentVariable"`	Index k (1-based) or 0 if convergent.
`"Dimension"`	n.
`"PolynomialExponents"`	$\{B_1, \ldots\}$, copied from spec.
`"MonoFactorLog"`	**[v3 new]** `{Const, Coeffs}`—tropical monomial factor data for `SplitRealImag` mode codegen (Fix B, Section 5.8).

For a divergent sector (`IsDivergent -> True`) the association contains the pre-flattening data only (no `FlattenedPolys`; `Prefactor` and `PrefactorBase` both hold $\lvert\det(M)\rvert$). The lifted path consumes this pre-flattened form before its own delta resolution. A *divergent lifted* sector (Section 5.7.4) likewise has `IsDivergent->True` with `NewExponents` the ε-carrying $\tilde{a}$, `DivergentVariable` the surviving slot k, `PrefactorBase` the lifted base, and a Case-A `DomainConstraint` (its $\text{ic}_k = 0$).

7.3 Lifted `SectorData` extensions (from `ProcessSectorLifted`)

Key	Description
`"HasConstantTerm"`	Whether the re-cleared $\tilde{Q}$ retains a constant term. `False` signals possible infinite true variance; the sample error bar may understate the true error.
`"EmptyDomain"`	(Alternative return) When the delta root lies entirely outside $[0,1]^n$; the sector contributes zero and is dropped.
`"PivotIndex"`	The pivot variable p eliminated against the delta constraint.
`"ZRow"`	The z-row m of the ray matrix M.
`"AugmentedA"`	Effective exponents of the $(n{+}1)$-dimensional problem before delta resolution.
`"Atilde"`	The $(n{-}1)$-vector of effective exponents after the monomial substitution.
`"DomainConstraint"`	Description of the feasible region of $[0,1]^n$ (compiled into the C++ integrand).

7.4 `LiftData` (from `LiftCoefficients`)

Key	Description
`"z0"`	Exact anchor value $z_0 = \lvert C_{\text{primary}}\rvert^{1/k}$ (kept exact in WL).
`"AuxVariable"`	`x[n+1]`.
`"AuxIndex"`	$n+1$.
`"Rules"`	List of lift rules applied.
`"Residuals"`	$c_i = C_i / z_0^{k_i}$ (exact, for multi-rule lifts).
`"OriginalSpec"`	Copy of the unlifted `IntegrandSpec`.

7.5 `DivergentSectorData` (from `ProcessDivergentSector`)

Key	Description
`"G0FlatPolys"`	$(n{-}1)$-dim flattened polynomials (simplified, $y_k = 0$).
`"G0Prefactor"`	Prefactor for the G_0 integral.
`"G0Dimension"`	$n-1$.
`"G1LogInsertions"`	`<|"VariableTerms"->...,"PolynomialTerms"->...|>`.
`"Remainder"`	`<|"FullPolys","SimplifiedPolys","Prefactor","DivVarIndex","DivVar`
`"AnalyticPole"`	$1/c_k$ (symbolic).
`"ck"`	Leading coefficient of $a_k(\varepsilon)$.

7.6 `IBPSectorData` (from `IBPProcessSector`)

Key	Description
`"ConeIndex"`	Integer label.
`"Method"`	`"IBP"`.
`"DivergentVariables"`	List of divergent variable indices.
`"NDivergent"`	Number of divergent variables.
`"ck"`	Leading coefficient c_k of $a_k(\varepsilon)$.
`"rk"`	Second-order correction $a_k^{(2)}/c_k$.
`"BoundaryData"`	`<|"FlatPolys","Prefactor","Dimension"(n-1),"PolyExponents","Log`
`"IBPTerms"`	List of term associations, each with: `"FlatPolys","Prefactor","Dimension"(n),"PolyExponents","Coeff0"`
`"NTerms"`	Total number of IBP terms.
`"IBPPrefactors"`	$\{-1/a_{k_1}, -1/a_{k_2}, \ldots\}$.
`"DomainConstraint"`	**[v3 new]** Lifted-sector domain indicator (`None` for unlifted); threaded into the boundary/term codegen (Section 5.7.4).
`"DLogPrefactor"`	**[v3 new]** $\partial_\varepsilon \log(\text{PrefactorBase})\|_0$; the driver adds its product with the pole to the finite part (0 for unlifted).
`"AnalyticPole"`	$1/c_k$ (symbolic).

8 Error Messages (`TropicalEval::*`)

Message	Fired by	Meaning
`degenerate`	`ProcessSector`	Cone with $\det(M) = 0$; sector returns `$Failed`.
`notsimplicial`	`ProcessSector`	Cone has $\neq n$ rays.
`nested`	`ProcessDivergentSector`	> 1 divergent variable; tropical subtraction refused.
`nestedIBP`	`IBPProcessSector`	> 1 divergent variable; IBP numerical assembly refused.
`badcpp`	`GenerateCpp`	Emitted C++ has unconverted heads or infinities; returns `$Failed`.
`nocuba`	driver/compiler	VEGAS requested but CUBA absent; returns `$Failed`.
`vegasbudget`	`resolveVegasSizing`	`NSamples` $< 20\cdot$`NStart`; error bar may be unreliable.
`validate`	validators	Sector sum deviates from `NIntegrate` beyond threshold.
`liftidentity`	`LiftCoefficients`	Round-trip check failed ($z \to z_0$ did not recover original).
`liftnopivot`	`ProcessSectorLifted`	No admissible pivot (lists per-pivot $\tilde{a}$).
`liftcomplex`	`ProcessSectorLifted`	Complex $\tilde{a}$ has no real domain indicator (`Direct` mode); use `ComplexExponentMode->"SplitRealImag"` for complex-B lifting (supported, incl. with a $1/\varepsilon$ pole).
`liftdivergent`	`ProcessSectorLifted`	(Convergent-lift backstop) selected pivot leaves $\mathrm{Re}(\tilde{a}_j) \leq 0$ with no regulator.
`liftdivdomain`	`ProcessSectorLifted`	**[v3 new]** Case B: the divergent variable couples to the lifted domain face ($\mathrm{ic}_k \neq 0$) for every admissible pivot; refused (log-space remap is future work).
`splitliftdiv`	driver	**[v3 new]** `SplitRealImag`×lift×divergence on the *inline symbolic-ε* Subtraction path (`Method->"None"/"Subtraction"`); use `Method->Automatic/"IBP"` or pinned-ε `LaurentFromSubtraction` (both supported, planCXLIFTDIV.md).
`splitdivmono`	`IBPProcessSector`	**[v3 new, planCXLIFTDIV.md]** Complex Case-B analogue: the dropped monomial factor carries a nonzero $\mathrm{Im}(B)$ exponent in the divergent direction ($\sum_j \mathrm{Im}(B_j) D_{j,k} \neq 0$), shifting the real pole; use the pinned-ε Subtraction route.
`liftfandim`	driver	Fan in lifted mode is not $(n{+}1)$-dimensional.
`liftdegenerate`	`EvaluateTropicalMCLifted`	Lifted Newton polytope is lower-dimensional; supply explicit `"FanData"`.

9 Limitations and Caveats

1. **Single divergent variable per sector.** Both divergence handlers (tropical subtraction and IBP) resolve a *single* divergent variable per sector. Sectors with > 1 divergent variable are detected and refused (`$Failed`, `TropicalEval::nested` / `::nestedIBP`). The symbolic reduction `IBPReduceSector` does iterate over several divergent variables, but the numerical boundary construction and multi-order Laurent assembly for $d \geq 2$ are not yet implemented. The full integral at a fixed, nonzero ε remains available via `"EpsilonValue"`. Enforced by `EXAMPLES/test_nested_divergence.wl`.

2. **Lifting + a simple $1/\varepsilon$ pole: supported (Case A); coupled cases refused.** A lifted sector may also carry a single $1/\varepsilon$ pole (Section 5.7.4): after the δ-resolution the pole sits in a surviving effective exponent $\tilde{a}_k$, which the IBP/subtraction machinery consumes; the sector carries a `PrefactorBase` and a `DomainConstraint`, and both the `LaurentFromSubtraction` (robust/universal) and `Method->"IBP"` routes resolve it (verified across $n = 2, 3, 4$ on non-trivial polynomials, cross-checks 41-C and 45). **Complex polynomial exponents B compose with lifting + a $1/\varepsilon$ pole** via `ComplexExponentMode->"SplitRealImag"` on *both* routes (planCXLIFTDIV.md, cross-check 46): the sector is decomposed on $\mathrm{Re}(B)$ (real measure, real divergence/domain machinery) and $\mathrm{Im}(B)$ rides as an oscillatory phase $\exp(i\,\mathrm{Im}(B)(\log P + \texttt{MonoFactorLog}))$. On the IBP route the per-IBP-piece `MonoFactorLog` is re-derived (un-flattened numerators divided by each piece's α_0) *and* the brought-down IBP coefficient carries the full complex B_j (the $i\,\mathrm{Im}(B_j)$ phase-derivative term); the IBP and Subtraction complex poles agree to $\sim 10^{-4}$ and match NIntegrate of the complex original ($n = 2, 3$). The single-pole-per-sector limit (item 1) still applies. Refused with a clean `$Failed` (never a wrong number): (i) **Case B**—the divergent variable couples to the lifted domain face ($\mathrm{ic}_k \neq 0$) for every admissible pivot (`liftdivdomain`; the Case-A-first pivot ranking makes this rare); (ii) `SplitRealImag`×lift×divergence on the *inline symbolic-ε* Subtraction path (`splitliftdiv`; use `Method->Automatic/"IBP"` or pinned-ε `LaurentFromSubtraction`); (iii) its complex Case-B analogue—a nonzero $\mathrm{Im}(B)$ monomial-factor exponent in the divergent direction (`splitdivmono`); (iv) `Direct`-mode complex-B lifting (`liftcomplex`—a complex $\tilde{a}$ has no real domain indicator; use `SplitRealImag`). When non-trivial polynomials produce `HasConstantTerm=False` sectors the per-sector IBP pole undercounts (item 3 mechanism); the Subtraction route is robust there and is the recommended route.

3. **`HasConstantTerm = False` ⇒ possible infinite MC variance (L5 caveat).** When the re-cleared polynomial $\tilde{Q}$ after delta-elimination lacks a constant term, the true variance of the lifted integrand may be infinite. The sample-based standard error *understates* the true error; the MC estimate can significantly deviate from the true value. Variance comparison between lifted and unlifted is unreliable for such sectors. *Always* cross-check against an independent reference (NIntegrate, CUBA Cuhre) and never accept a sample error bar from a `HasConstantTerm=False` sector without corroboration. This situation arises structurally for certain degenerate lifted polytopes (e.g. Test 23C, polynomial with extreme geometry).

4. **`SplitRealImag Re(B)` caveat.** The opt-in `"SplitRealImag"` mode builds VEGAS importance sampling from $\mathrm{Re}(B_j)$. For very large $|\mathrm{Im}(B_j)|$ this can inflate variance relative to `"Direct"`. *Always* compare `SplitRealImag` against `Direct` and against an independent reference before trusting the result. The default `"Direct"` mode is always safe.

5. **High-D VEGAS error bars can lie.** VEGAS defaults (`NStart=1000, NIncrease=500, NBatch=1000`) produce confidently wrong values for sector dimension $\gtrsim 6$. `resolveVegasSizing` and `vegasbudget` mitigate this, but in high dimension always cross-check rather than

trusting the VEGAS error bar. (Observed: 8D unlifted 1.2% off at 7σ; lifted $\sim$45% off at default budget.)

6. **Degenerate lifted polytope.** When the lifted polynomial has fewer monomials than $(n{+}2)$ in $(n{+}1)$ variables, or those monomials lie in a lower-dimensional affine subspace, the automatic fan fails (`liftdegenerate`). Supply an explicit `"FanData"`. Such polynomials may also structurally produce all-`HasConstantTerm=False` sectors (see item 3).

7. **Kinematic-symbol coefficients cannot be auto-lifted.** `DetectExtremeCoefficients` skips kinematic (symbolic) coefficients since their magnitude is unknown at lift time. Manual lift rules are still allowed.

8. **No multiple independent auxiliary variables.** A single z with per-monomial powers and residual coefficients handles multiple extreme coefficients (the k^* rule). Multiple independent auxiliary variables are future work.

9. **Shared `INTERFILES/` directory.** The C++ MC writes fixed filenames (`tropical_mc`, `tropical_mc_generated.cpp`, `mc_results.txt`). Run MC-executing scripts one at a time per `WorkingDirectory`; two concurrent MC jobs clobber each other's files and yield bogus numbers. The symbolic, NIntegrate-gated test suite is unaffected.

10. **Double precision.** All C++ computation uses 64-bit doubles. Higher precision requires additional libraries.

11. **FIESTA cross-checks require FIESTA and path edits.** The `CC/` scripts (`compare_pole_and_finite.wl`, etc.) require an independent FIESTA5 installation and path configuration. They skip gracefully if FIESTA is absent.

10 Worked Examples

10.1 v3 Capstone: Divergent + Complex Exponents (Ex-V3a)

A one-loop integral with an ε-pole and complex exponents:

```
Get["tropical_eval.wl"];

spec = <|
  "Polynomials"      -> {1 + x[1] + x[2] + x[1]*x[2]},
  "MonomialExponents" -> {2*eps - 1, 0},
  "PolynomialExponents"-> {-2 + I/3},    (* complex B *)
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> eps
|>;

verts  = PolytopeVertices[(1 + x[1] + x[2] + x[1]*x[2])^(-1),
                          {x[1], x[2]}];
fanData = ComputeDecomposition[verts];

(* IBP default, VEGAS sampler *)
results = EvaluateTropicalMC[spec, fanData, {},
           "Method"    -> "IBP",
           "Integrator" -> "VEGAS",
           "EpsilonValue" -> 0.01];
(* Cross-check with subtraction -- they must agree on (pole, finite) *)
ref = LaurentFromSubtraction[spec, fanData, {}];
```

Cross-checks: (pole, finite) via IBP vs subtraction (#4), vs exact Γ-product Laurent (#3), vs NIntegrate (#1).

10.2 v3 Capstone: Small Complex Coefficient, Lifted + VEGAS (Ex-V3b)

```
(* Complex coefficient: the bug-1 regime *)
poly = 1 + (1*^-4 + 2*^-4*I)*x[1]^2 + x[2]^2 + x[1]*x[2]^2;
spec = <|
  "Polynomials"      -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents"-> {-2 + I/2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> None
|>;

(* DetectExtremeCoefficients finds the 1e-4+2e-4*I monomial;
   EvaluateTropicalMCLifted auto-lifts it at k* *)
results = EvaluateTropicalMCLifted[spec, {},
           "Integrator"        -> "VEGAS",
           "ComplexExponentMode" -> "SplitRealImag"];
(* Check 1: SplitRealImag == Direct *)
ref = EvaluateTropicalMCLifted[spec, {},
       "Integrator"        -> "VEGAS",
       "ComplexExponentMode" -> "Direct"];
(* Cross-check 2: vs CUBA Cuhre on the direct integrand (#21b) *)
```

This is the case that exposed BUG 1: complex coefficients with $\arg P \neq 0$ using `SplitRealImag`. The fix (Section 5.8, Fix A) is verified here.

10.3 v3 Capstone: Kinematic Scan, Batched VEGAS (Ex-V3c)

```
poly = 1 + lam*x[1]^2 + x[2]^2 + x[1]*x[2]^2;
spec = <|
  "Polynomials"      -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents"-> {-2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {lam},
  "RegulatorSymbol"  -> None
|>;

verts   = PolytopeVertices[(1 + x[1]^2 + x[2]^2 + x[1]*x[2]^2)^(-1),
                           {x[1], x[2]}];
fanData = ComputeDecomposition[verts];

kinPoints = Table[{0.5 + 0.01 i}, {i, 200}];

(* One compile, one run -- batch shares one LDS sample set *)
results = EvaluateTropicalMC[spec, fanData, kinPoints,
           "Integrator" -> "VEGAS",
           "Batch"      -> True,
           "NSamples"   -> 2*^6];
(* Verify: per-kp VEGAS result must agree within combined error (#23) *)
```

10.4 Minimal Convergent Integral

```
Get["tropical_eval.wl"];

poly = 1 + 2*x[1]^2 + x[2]^2 + x[1]*x[2]^2;
vars = {x[1], x[2]};
spec = <|
  "Polynomials"      -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents"-> {-2},
  "Variables"        -> vars,
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> None
|>;

verts   = PolytopeVertices[poly^(-1), vars];
fanData = ComputeDecomposition[verts];

vr = ValidateDecomposition[spec, fanData, {}, 3];
Print["Relative error: ", vr["RelativeError"]];

results = EvaluateTropicalMC[spec, fanData, {}];
```

10.5 Divergent Integral with Regulator

```
spec = <|
  "Polynomials"      -> {1 + x[1] + x[2] + x[1]*x[2]},
  "MonomialExponents" -> {2*eps - 1, 0},
  "PolynomialExponents"-> {-2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> eps
|>;

verts   = PolytopeVertices[
           (1 + x[1] + x[2] + x[1]*x[2])^(-1), {x[1], x[2]}];
fanData = ComputeDecomposition[verts];

(* IBP default, compute at eps=0.01 *)
results = EvaluateTropicalMC[spec, fanData, {},
           "EpsilonValue" -> 0.01];
(* Result: pole coefficient (analytic) + finite part (MC) + remainder (MC) *)
```

10.6 Lifting a Small Coefficient

```
poly = 1 + 10^(-4)*x[1]^2 + x[2]^2 + x[1]*x[2]^2;
spec = <|
  "Polynomials"      -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents"-> {-2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> None
```

```
|>;

(* Inspect what would be lifted *)
extremes = DetectExtremeCoefficients[spec];
Print[extremes]; (* SuggestedK uses k* rule *)

(* Auto-lift and evaluate *)
results = EvaluateTropicalMCLifted[spec, {},
           "Integrator" -> "VEGAS",
           "NSamples"  -> 2*^6];

(* Verify: ValidateLiftedDecomposition must match NIntegrate to < 0.5% *)
```

The HasConstantTerm field is reported per sector; check for False sectors and cross-validate against CUBA Cuhre if any are present.

10.7 Lifting a Divergent Integral (Case A)

An integrand that needs lifting (an extreme coefficient) *and* has a $1/\varepsilon$ pole. The denominator carries $10^6 x_1^2$ (triggers lifting) and the monomial $x_1^{-1+\varepsilon}$ gives a simple pole at $x_1 \to 0$. The pole is $\int_0^\infty (1+x_2^2)^{-2} dx_2 = \pi/4$.

```
spec = <|
  "Polynomials"      -> {1 + x[1]*x[2] + 10^6*x[1]^2 + x[2]^2},
  "MonomialExponents" -> {-1 + eps, 0},  (* x1^{-1+eps}: the pole *)
  "PolynomialExponents"-> {-2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> eps
|>;
liftRules = {<|"PolyIndex" -> 1, "ExponentVector" -> {2, 0}, "k" -> 3|>}; (* z0
    ↪ =100 *)

(* IBP route: pole + finite directly *)
ibp = EvaluateTropicalMCLifted[spec, {{}}, "LiftRules" -> liftRules,
       "Method" -> "IBP", "NSamples" -> 5*^5];
(* ibp["Results"][[1]]: PoleCoefficient ~ pi/4, FinitePart *)

(* Subtraction route (robust/universal): build the lifted spec + fan, fit Laurent
    ↪  *)
lift = LiftCoefficients[spec, liftRules];
fan  = computeFanScaled[PolytopeVertices[
        (Times @@ lift["LiftedSpec"]["Polynomials"])^(-1),
        lift["LiftedSpec"]["Variables"]]];
sub  = LaurentFromSubtraction[lift["LiftedSpec"], fan, {{}},
        "LiftData" -> lift["LiftData"], "EpsilonValues" -> {0.01, 0.02, 0.04}];
(* sub["Results"][[1]]: Pole ~ pi/4, Finite; agrees with the IBP route *)
```

Both routes give `pole` $\approx \pi/4$ and agreeing finite parts, and reproduce `NIntegrate` of the original at small ε to $< 0.3\%$ (cross-check 41-C). For non-trivial polynomials that produce `HasConstantTerm=False` sectors, prefer the Subtraction route (it sums the full decomposition and is robust; cross-check 45 covers $n = 2, 3, 4$). See Section 5.7.4.

11 Validation and Testing

11.1 V3 Cross-Check Suite (TEST/, 45 named checks)

The suite is organized by plan.md §8. Key checks:

#	What it checks
1	Sector sum vs direct `NIntegrate` (all examples; relerr $< 10^{-3}$).
2	vs exact Gamma/Beta/Dirichlet closed form; complex A_i and B_j.
3	Divergent (pole, finite) vs exact Γ-product Laurent; B=4 → $(1/6, -1/6)$, B=5 → $(1/12, -1/8)$.
4	IBP (default) vs subtraction agree on (pole, finite) to $\lesssim 10^{-3}$.
5	vs CUBA Cuhre on the direct integrand (Tier 2).
8	vs FIESTA5 on bubblepp presector (Tier 3, needs FIESTA).
17	Lifted vs unlifted exactness (relerr $< 0.5\%$, `DroppedSectors` asserted).
19	Sample σ vs true σ; flags `HasConstantTerm=False` sectors.
21b	Complex-BASE + complex-exponent lifting ($\arg P \neq 0$); gates the BUG 1 fix.
22	Determinism: same seed → bit-identical; distinct seeds → distinct streams within 5σ.
23	Batched VEGAS = per-kp VEGAS within combined error (gates the §5.1 fold).
25	Golden-master codegen regression (byte-identical emitted C++ across all integrand types).
27	Known-limitation enforcement: `nested/nestedIBP/nocuba/badcpp/liftnopivot/liftcom` all return `$Failed` cleanly.
31	Merge fidelity vs Tree A: all Tree-A goldens reproduced.
32	Merge fidelity vs Tree B: all Tree-B goldens reproduced.
33	Divergence × complex exponents, full matrix: {IBP, subtraction} × {MC, VEGAS}.
34	Lifting × complex coefficient × VEGAS, high-D (≈8D); gates BUG 1 + L1/L2/L3.
40	Lift identity (symbolic): $z \to z_0$ recovers original, exact, for +, −, complex, multi-rule.
41	Numerator ($B > 0$) × {divergence, lifting, **lifting×divergence**}. 41-C: lifted + $1/\varepsilon$ pole; IBP and Subtraction poles/finites agree and reproduce `NIntegrate`; pole = $\pi/4$.
43	Exactness guard: `FreeQ[symbolicOutput, _Real]` before the `MmaToC` boundary. Sub-case D: lifted+divergent exact input ($\tilde{a}$, `PrefactorBase`, IBP boundary/terms, `DLogPrefactor`).
44	**Case-B refusal** + safety invariant: every *emitted* divergent lifted sector is decoupled (Case A); `liftdivdomain` is the wired clean `$Failed` for the coupled case.
45	**Lift × divergence battery**, $n = 2, 3, 4$, non-trivial polynomials (cross terms, numerators, multiple polynomials): Subtraction route reconstructs `NIntegrate` to $< 0.3\%$; IBP agrees on `HasConstantTerm`-clean cases.

Adversarial verification principle.

No gate is judged by the agent that produced the result. Each numeric PASS is checked by a separate verifier agent against ≥ 2 independent oracles. The complex-lifting phases run a 3-lens

panel (exact symbolic, independent integrator, reseed/budget stability); all three must confirm. See plan.md §8.5.

11.2 Examples 1–23 (from Tree A and Tree B)

The example suite (`EXAMPLES/`) covers:

- Examples 1–14: convergent core (basic 2D, complex exponents, kinematic scan, tropical factoring, MmaToC, C++ pipeline, multiple polynomials, numerator factors, 3D/4D).
- Examples 15–20: complex exponents and small/extreme coefficients; lifting; CheckFlattenMagnitude.
- Examples 21–23: exact Gamma/Beta cross-check; CUBA Cuhre cross-check; Test 23 (lifted pipeline, subtests A/B/C/D).
- Ex-V3a–d: new v3 capstone examples (divergent+complex, lifted+complex+VEGAS, batched scan, numerator+pole).

11.3 FIESTA Cross-Checks (`CC/`)

Cross-validation against FIESTA5 on:

- Simple tests: $\pi/8$ integral (convergent, exact), Gamma integral (divergent pole exact), 2D with kinematics.
- Full 5D cosmological integral (bubblepp presector 1): 36 convergent + 14 divergent sectors, all $c_k = 2/7$, compared to FIESTA to within 2σ (GL(1)/Cheng–Wu gauge transformation validated).

Requires FIESTA5 installation; scripts skip gracefully if absent.

12 Why the Pipeline Works

Tropical factoring is exact.

The identity $P(\mathbf{y}) = \mathbf{y}^d \cdot Q(\mathbf{y})$ is algebraic, not a numerical approximation. No numerical error is introduced by the factoring step; the correctness of the decomposition is independent of coefficient values.

Flattening is a change of variables.

The substitution $y_i = (y_i')^{1/a_i}$ is an exact coordinate change. The y^{a-1} singularity is absorbed completely into the measure, not approximated.

Lifting is an identity.

The replacement $C\mathbf{x}^\alpha \to z^k \mathbf{x}^\alpha$ with anchor $z_0 = |C|^{1/k}$ followed by $\delta(z - z_0)$ integration recovers the original integral exactly. The round-trip identity $z \to z_0$ is verified symbolically by `LiftCoefficients`. What changes is only the geometry: the fan of the lifted polynomial adapts to the extreme coefficient.

Pole extraction is correct.

The $1/(c_k\varepsilon)$ pole from $\int_0^1 y_k^{c_k\varepsilon - 1} dy_k = 1/(c_k\varepsilon)$ is exact. The subtraction $f - f|_{y_k=0}$ vanishes at $y_k = 0$, making the remainder integral convergent. The IBP boundary at $y_k = 0$ vanishes for $\mathrm{Re}(a_k) > 0$ (i.e. for $\varepsilon > 0$). Both methods are independently correct and their agreement to $\lesssim 10^{-3}$ on (pole, finite) is a cross-check.

The exactness invariant enables unattended correctness gates.

Because the decomposition is exact, the verifier checks it with exact symbolic identities (lift round-trip, tropical factoring identity, det M and exponent bookkeeping, IBP/subtraction pole identity, GL(1) chart identity)—all decisive, none statistical. The single genuinely-numeric output (the integrated value) is gated against ≥ 2 independent oracles. This is what makes unattended phase gates trustworthy.

13 Dependencies

Polymake (https://polymake.org) Polyhedral computation. Must be installed and accessible on `PATH` or in `/opt/homebrew`, `/usr/local`, or `/usr`.

g++ (GCC C++ compiler) C++17 required (`-std=c++17`). OpenMP recommended (`-fopenmp`); auto-retried without it on macOS.

Wolfram Mathematica (version 12.0+) Required for all `.wl` files.

CUBA (*optional*) (https://feynarts.de/cuba/) Required only for `"Integrator" -> "VEGAS"`. Auto-detected; install via `brew install cuba` or set `CUBA_INCLUDE`/`CUBA_LIB`. Absent ⇒ plain MC unaffected; VEGAS fails fast with an actionable message.

FIESTA5 (*optional*) Required only for `CC/` cross-check scripts.

14 Command-Line Execution

```
# Full validation suite (exits nonzero on any FAIL)
wolframscript -file EXAMPLES/run_validation_suite.wl

# Divergent pipeline tests
wolframscript -file EXAMPLES/test_divergent.wl

# Lifted pipeline tests (Test 23 A/B/C/D)
wolframscript -file EXAMPLES/test_lifted.wl

# IBP + subtraction cross-check (3D divergent, all 4 combos)
wolframscript -file EXAMPLES/test_divergent_crosscheck.wl

# MC vs VEGAS parity (requires CUBA)
wolframscript -file EXAMPLES/test_vegas.wl

# CUBA cross-check of all test-suite integrals (requires CUBA)
wolframscript -file EXAMPLES/test_cuba.wl

# SeedBase runtime-argv override test
wolframscript -file EXAMPLES/test_seedbase.wl

# Nested-divergence limitation pinned (clean $Failed)
wolframscript -file EXAMPLES/test_nested_divergence.wl

# Complex exponents + small coefficients (Examples 15-20)
wolframscript -file EXAMPLES/tropical_eval_examples2.wl

# Exact Gamma + CUBA cross-checks (Examples 21-23)
wolframscript -file EXAMPLES/tropical_eval_examples3.wl
```

```
# Full cross-check suite entry point
wolframscript -file TEST/run_all_checks.wl
```

IMPORTANT: Run scripts that execute the C++ Monte Carlo *one at a time* per `WorkingDirectory`—two concurrent MC jobs clobber each other's generated files.